

GESTION DU PERSONNEL DES LIGUES

Documentation Technique du Projet

Projet	Gestion du Personnel des Liges
Version	2.0
Langage	Java 8
Outil de build	Maven
Base de donnees	MySQL 8.0
Groupe	Reda Mekaoui (Team leader), Milhane Moualfi, Valentin Semanchuk, Kikouedi Dandou Messi
Annee	2025 / 2026

Projet encadre dans le cadre du cours DCP

Sommaire

1. Contexte du Projet

- 1.1 Objectifs fonctionnels
- 1.2 Organisation pédagogique

2. Architecture du Projet

- 2.1 Structure des packages
- 2.2 Couches applicatives
- 2.3 Diagramme de classes simplifié

3. Description des Classes

- 3.1 Package personnel
- 3.2 Package jdbc
- 3.3 Package serialisation

4. Interface en Ligne de Commande

- 4.1 Menus principaux
- 4.2 Arbre de navigation
- 4.3 Gestion des dates dans la console

5. Base de Données

- 5.1 Modèle Conceptuel des Données (MCD)
- 5.2 Schéma MySQL
- 5.3 Gestion des rôles
- 5.4 Paramètres de connexion

6. Tests Unitaires

- 6.1 Couverture des tests
- 6.2 Cas particuliers testés
- 6.3 Lancement des tests

7. Dépendances et Lancement

- 7.1 Dépendances Maven
- 7.2 Prérequis
- 7.3 Étapes de lancement

8. Maquettes de l'Interface Graphique

1. Contexte du Projet

Ce projet a été réalisé dans le cadre d'un travail encadré. Il s'agit d'une application Java permettant de gérer le personnel de différentes ligues sportives hébergées par la M2L (Maison des Ligues de Lorraine).

L'application permet à un super-utilisateur (root) de gérer l'ensemble des ligues et de leurs employés. Chaque ligue peut avoir un administrateur désigné parmi ses employés, qui dispose de droits de gestion sur sa propre ligue.

Le projet a été développé en groupe de 4, avec des tâches assignées par l'enseignant via une plateforme de gestion de projet pédagogique. Chaque membre était évalué individuellement sur ses commits Git.

1.1 Objectifs fonctionnels

- Gérer des ligues sportives : création, renommage, suppression
- Gérer les employés de chaque ligue : ajout, modification, suppression
- Définir des administrateurs par ligue avec droits restreints
- Persister les données via JDBC (MySQL) ou sérialisation Java
- Valider la cohérence des dates d'arrivée et de départ des employés
- Garantir l'intégrité du système (impossible de supprimer le root)

1.2 Organisation pédagogique

Le professeur assignait des tâches précises à réaliser, chacune correspondant à une fonctionnalité ou correction. Les contributions étaient suivies via des commits Git, qui servaient de base à l'évaluation individuelle de chaque membre du groupe.

Personnel 1 - Dates

✓ Terminé : Consulter ✓ Terminé : Remettre un travail ✓ Terminé : Recevoir une note ✓ Terminé : Obtenir une note minimale de réussite

À rendre : lundi 3 novembre 2025, 14:00

Nous souhaitons que les employés aient une date d'arrivée et une date de départ.

- Modifiez le MCD en conséquence
- Modifiez la couche métier : variables d'instance de type `LocalDate`, constructeur, getters, setters
- En cas de dates incohérentes, déclenchez des exceptions
- Vérifiez le bon fonctionnement des dates en ajoutant des tests unitaires

Déposez les commits contenant ces informations ci-dessous.

Figure 1 - Tâche assignée : Personnel 1 - Dates (ajout des dates d'arrivée et de départ)

Personnel 1 - Tests unitaires

✓ Terminé : Consulter À faire : Remettre un travail ✓ Terminé : Recevoir une note ✗ Échec : Obtenir une note minimale de réussite

À rendre : lundi 3 novembre 2025, 14:00

Vérifiez le fonctionnement de tous les setters en ajoutant des tests unitaires à ceux déjà présents.

N'oubliez pas de vérifier ce qui se passe quand on supprime un objet ou que l'on modifie l'administrateur d'une ligue.

Déposez les adresses des ressources concernées.

Figure 2 - Tâche assignée : Personnel 1 - Tests unitaires (vérification des setters et suppression)

2. Architecture du Projet

2.1 Structure des packages

Le projet suit une architecture en couches separees, chacune encapsulee dans un package Java distinct :

```
src/
|- commandLine/          # Interface en ligne de commande (CLI)
|   |- PersonnelConsole.java
|   |- LigueConsole.java
|   `-- EmployeConsole.java
|- jdbc/                 # Acces a la base de donnees via JDBC
|   |- JDBC.java
|   |- Credentials.java
|   `-- CredentialsExample.java
|- personnel/            # Logique metier (couche domaine)
|   |- GestionPersonnel.java
|   |- Ligue.java
|   |- Employe.java
|   |- Passerelle.java
|   `-- [Exceptions...]
|- serialisation/
|   `-- Serialization.java
`-- testsUnitaires/
    |- testEmploye.java
    `-- testLigue.java
```

2.2 Couches applicatives

Couche	Package	Role
Presentation	commandLine	Menus interactifs en ligne de commande
Domaine / Metier	personnel	Entites, regles metier, exceptions
Persistence JDBC	jdbc	Connexion et operations CRUD sur MySQL
Persistence fichier	serialisation	Serialisation binaire Java (.srz)
Tests	testsUnitaires	Tests JUnit 5 sur les entites

2.3 Diagramme de classes simplifie

```
GestionPersonnel (Singleton)
    |- root : Employe
    `-- ligues : SortedSet<Ligue>

Ligue
    |- nom : String
    |- id : int
    |- administrateur : Employe
    `-- employes : SortedSet<Employe>

Employe
    |- nom, prenom, mail, password : String
    |- dateArrivee, dateDepart : LocalDate
```

```
`- ligue : Ligue
```

```
Passerelle (interface)
```

```
| - JDBC      -> implementation MySQL
```

```
`- Serialization -> implementation fichier binaire
```

3. Description des Classes

3.1 Package personnel

GestionPersonnel

Classe principale du projet, implementee selon le patron de conception Singleton. Elle est le point d'entree unique pour acceder a l'ensemble des donnees (ligues, root) et delegue les operations de persistance a la passerelle configuree.

```
// Choix du mode de persistance
public final static int SERIALIZATION = 1, JDBC = 2,
    TYPE_PASSELLE = JDBC;
```

Ligue

Represente une ligue sportive. Contient un ensemble trie d'employes et un administrateur. Par default, le root est administrateur de toutes les ligues tant qu'aucun autre n'est designe.

Methode	Description
getNom() / setNom(nom)	Lecture/modification du nom de la ligue
getAdministrateur()	Retourne l'employe administrateur
setAdministrateur(employe)	Designe un nouvel administrateur (avec mise a jour BDD)
addEmploye(...)	Cree et ajoute un employe a la ligue
remove(employe)	Supprime un employe de la ligue
getEmployes()	Retourne les employes (tri alphabetique)

Employe

Represente un employe d'une ligue. Ne peut pas etre instancie directement : il faut passer par Ligue.addEmploye(...). Chaque employe possede des dates d'arrivee et de depart, soumises a validation de coherence.

Methode	Description
getNom() / setNom(nom)	Getter/Setter du nom
getMail() / setMail(mail)	Getter/Setter du mail
getPassword() / setPassword(mdp)	Getter/Setter du mot de passe
getDateArrivee() / setDateArrivee(date)	Date d'arrivee (ne peut pas etre dans le futur)
getDateDepart() / setDateDepart(date)	Date de depart (ne peut pas etre avant l'arrivee)
estAdmin(ligue)	Verifie si l'employe est admin de la ligue
estRoot()	Verifie si le compte est le super-utilisateur
remove()	Supprime le compte (root recupere les droits admin si besoin)

Passerelle (interface)

Definit le contrat pour toute couche de persistance. Deux implementations existent : JDBC (MySQL) et Serialization (fichier binaire). La constante TYPE_PASSERELLE dans GestionPersonnel determine laquelle est utilisee.

```
GestionPersonnel getGestionPersonnel();  
void sauvegarderGestionPersonnel(GestionPersonnel gp) throws  
SauvegardeImpossible;  
int insert(Ligue ligue) throws SauvegardeImpossible;  
int insert(Employe employe) throws SauvegardeImpossible;  
int update(Ligue ligue) throws SauvegardeImpossible;  
int update(Employe employe) throws SauvegardeImpossible;  
int delete(Employe employe) throws SauvegardeImpossible;
```

Exceptions personnalisées

Classe	Description
DateInvalide	Date de depart anterieure a l'arrivee, ou arrivee dans le futur
DroitsInsuffisants	Action tentee sans les droits necessaires (RuntimeException)
ImpossibleDeSupprimerRoot	Tentative de suppression du super-utilisateur (RuntimeException)
SauvegardeImpossible	Echec de la sauvegarde JDBC ou par serialisation

3.2 Package jdbc

JDBC

Implementation de l'interface Passerelle via JDBC. Gere la connexion a MySQL au demarrage et toutes les operations CRUD. Chargement des ligues et employes au demarrage, avec restauration du role administrateur.

Note de securite : ne jamais committer Credentials.java avec des identifiants reels. Utiliser CredentialsExample.java comme modele.

3.3 Package serialisation

Serialization

Implementation alternative de Passerelle utilisant la serialisation Java. Persiste l'integralite du graphe d'objets GestionPersonnel dans un fichier binaire (GestionPersonnel.srz). Avantage : aucune base de donnees requise. Inconvenient : moins robuste, inadapté a des acces concurrents.

4. Interface en Ligne de Commande

L'interface utilisateur est entièrement textuelle et repose sur la bibliothèque CommandLineMenus. Elle organise la navigation en menus hiérarchiques avec sélection par saisie d'une lettre ou d'un code.

4.1 Menus principaux

Classe	Responsabilité
PersonnelConsole	Menu principal : root, ligues, quitter (avec/sans sauvegarde)
LigueConsole	Gestion des ligues : lister, ajouter, sélectionner, renommer, supprimer
EmployeConsole	Gestion des employes : afficher, modifier champs, supprimer

4.2 Arbre de navigation

Le schéma ci-dessous illustre l'ensemble des menus accessibles depuis le point d'entrée principal, ainsi que les enchainements de navigation possibles.

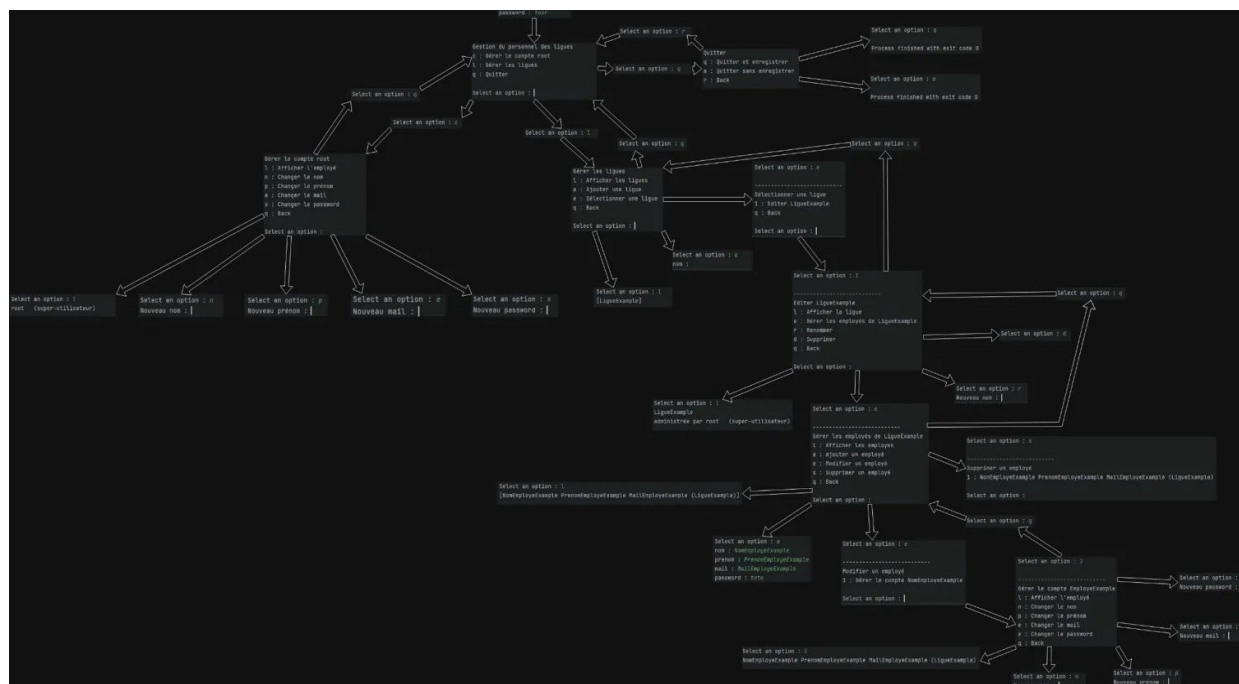


Figure 3 - Arbre complet de navigation de l'interface console

4.3 Gestion des dates dans la console

La saisie des dates suit le format ISO (yyyy-mm-dd). En cas de format incorrect, un message d'erreur explicite est affiché. En cas de date incohérente, l'exception `DateInvalide` est interceptée et un message est présenté à l'utilisateur.

```
// Exemple dans EmployeConsole.java
String strda = getString("Nouvelle date d'arrivee (yyyy-mm-dd): ");
employe.setDateArrivee(LocalDate.parse(strda));
// En cas d'erreur : DateTimeParseException ou DateInvalide
```

5. Base de Donnees

5.1 Modele Conceptuel des Donnees (MCD)

Le MCD presente deux entites principales : LIGUE et EMPLOYE, liees par une relation Appartient de cardinalite 0,n - 0,1. Un employe appartient a au plus une ligue ; une ligue peut avoir de 0 a n employes.

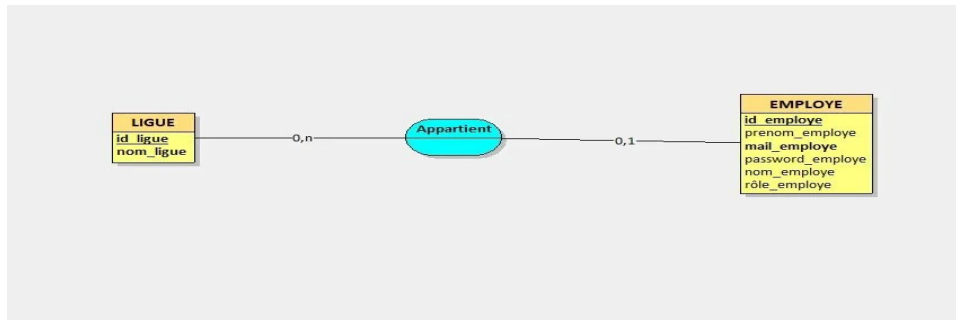


Figure 4 - MCD : entites LIGUE et EMPLOYE avec la relation Appartient

5.2 Schema MySQL

```

CREATE TABLE LIGUE (
    id_ligue    INT AUTO_INCREMENT PRIMARY KEY,
    nom_ligue   VARCHAR(255) NOT NULL
);

CREATE TABLE EMPLOYE (
    id_employe  INT AUTO_INCREMENT PRIMARY KEY,
    nom_employe VARCHAR(255),
    prenom_employe VARCHAR(255),
    mail_employe VARCHAR(255),
    password_employe VARCHAR(255),
    date_arrivee_employe DATE,
    date_depart_employe VARCHAR(255),
    role_employe VARCHAR(50), -- "admin" ou "employe"
    id_ligue    INT,
    FOREIGN KEY (id_ligue) REFERENCES LIGUE(id_ligue) ON DELETE CASCADE
);
  
```

5.3 Gestion des roles

Un champ role_employe (valeurs : "admin" ou "employe") est maintenu en base de donnees pour chaque employe. Ce champ est mis a jour automatiquement lors des operations setAdministrateur() et lors de la suppression d'un administrateur.

Le root (super-utilisateur) est identifiable par l'absence de valeur dans id_ligue (NULL).

5.4 Parametres de connexion

Parametre	Valeur
Host	localhost
Port	3306
Base de donnees	personnel
Driver	com.mysql.cj.jdbc.Driver
Utilisateur / Mot de passe	A configurer dans Credentials.java

6. Tests Unitaires

Les tests sont écrits avec JUnit 5 et couvrent les fonctionnalités des classes `Employe` et `Ligue`. Ils sont situés dans le package `testsUnitaires`.

6.1 Couverture des tests

Fichier de test	Classe testée	Tests inclus
testEmploye.java	Employe	getNom, setNom, getPrenom, setPrenom, getMail, setMail, setPassword, getLigue, getDateArrivee, setDateArrivee, getDateDepart, setDateDepart, remove
testLigue.java	Ligue	createLigue, addEmploye, ExceptionDateInvalideDA, ExceptionDateInvalideDD, setNom, getNom, TestChangementAdmin, TestRemoveLigue, TestRemoveAdmin

6.2 Cas particuliers testés

- Levée de `DateInvalide` si la date d'arrivée est postérieure à la date actuelle
- Levée de `DateInvalide` si la date de départ est antérieure à la date d'arrivée
- Suppression d'un administrateur : le root récupère automatiquement les droits
- Suppression d'un employé : il est retiré de la liste des employés de sa ligue
- Changement d'administrateur : vérification que le nouvel admin est bien enregistré

6.3 Lancement des tests

```
mvn test
```

7. Dependencies et Lancement

7.1 Dependances Maven

Dependance	Version	Role
mysql-connector-java	8.0.16	Driver JDBC pour la connexion a MySQL
CommandLineMenus	2.0	Bibliotheque de menus interactifs en console
junit-jupiter-api	5.8.2	Framework de tests unitaires JUnit 5
junit-jupiter-engine	5.8.2	Moteur d'execution JUnit 5

7.2 Prerequis

- Java 8 ou superieur
- Apache Maven
- MySQL 8.0 (ou mode serialisation pour fonctionner sans base de donnees)

7.3 Etapes de lancement

1. Cloner le depot :

```
git clone <url_du_depot>
```

2. Configurer la base de donnees :

- Creer la base de donnees personnel dans MySQL
- Executer le script SQL de creation des tables (cf. section 5.2)
- Renseigner les identifiants dans Credentials.java (ne pas committer)

3. Compiler le projet :

```
mvn compile
```

4. Lancer l'application :

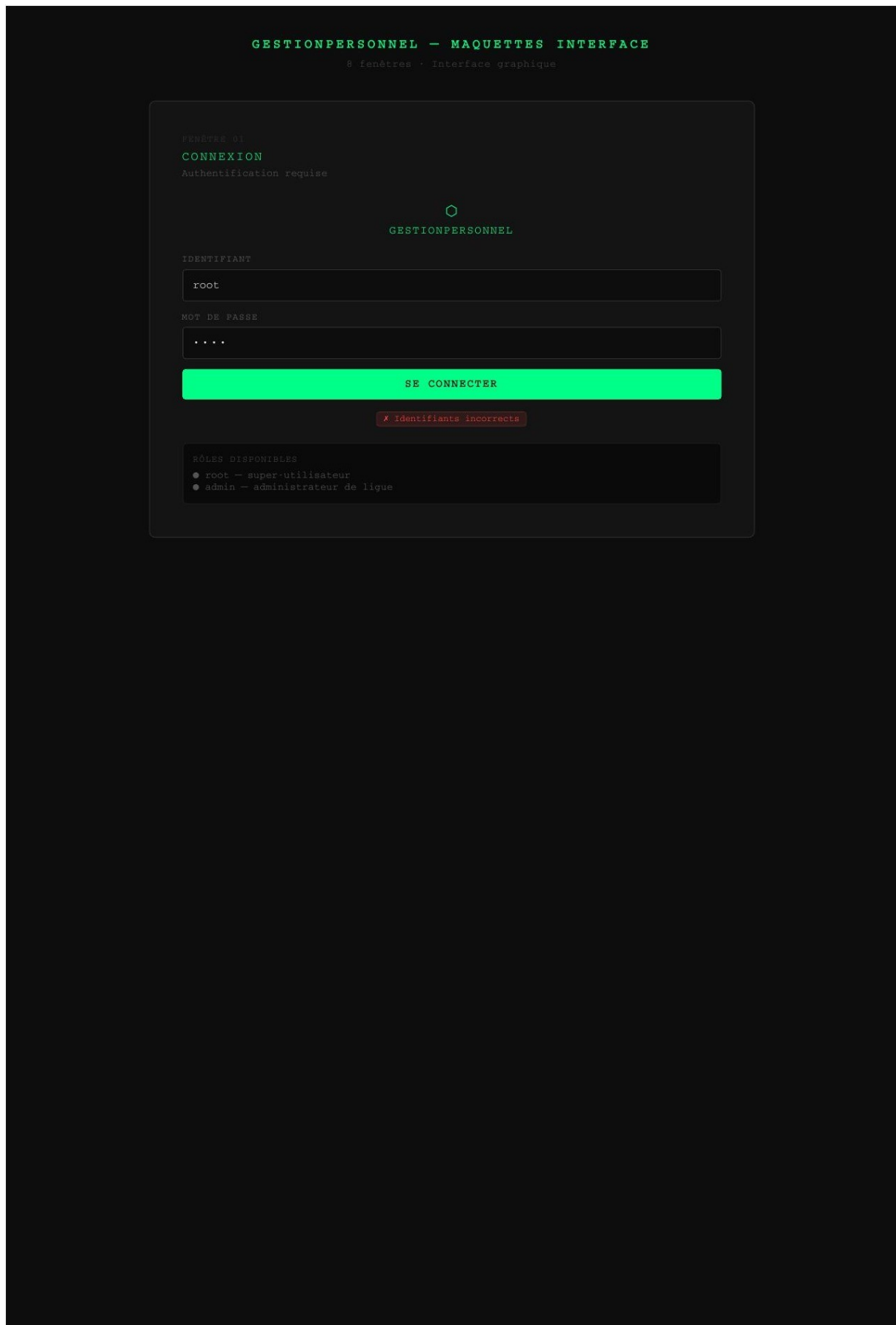
```
mvn exec:java
```

5. Lancer les tests :

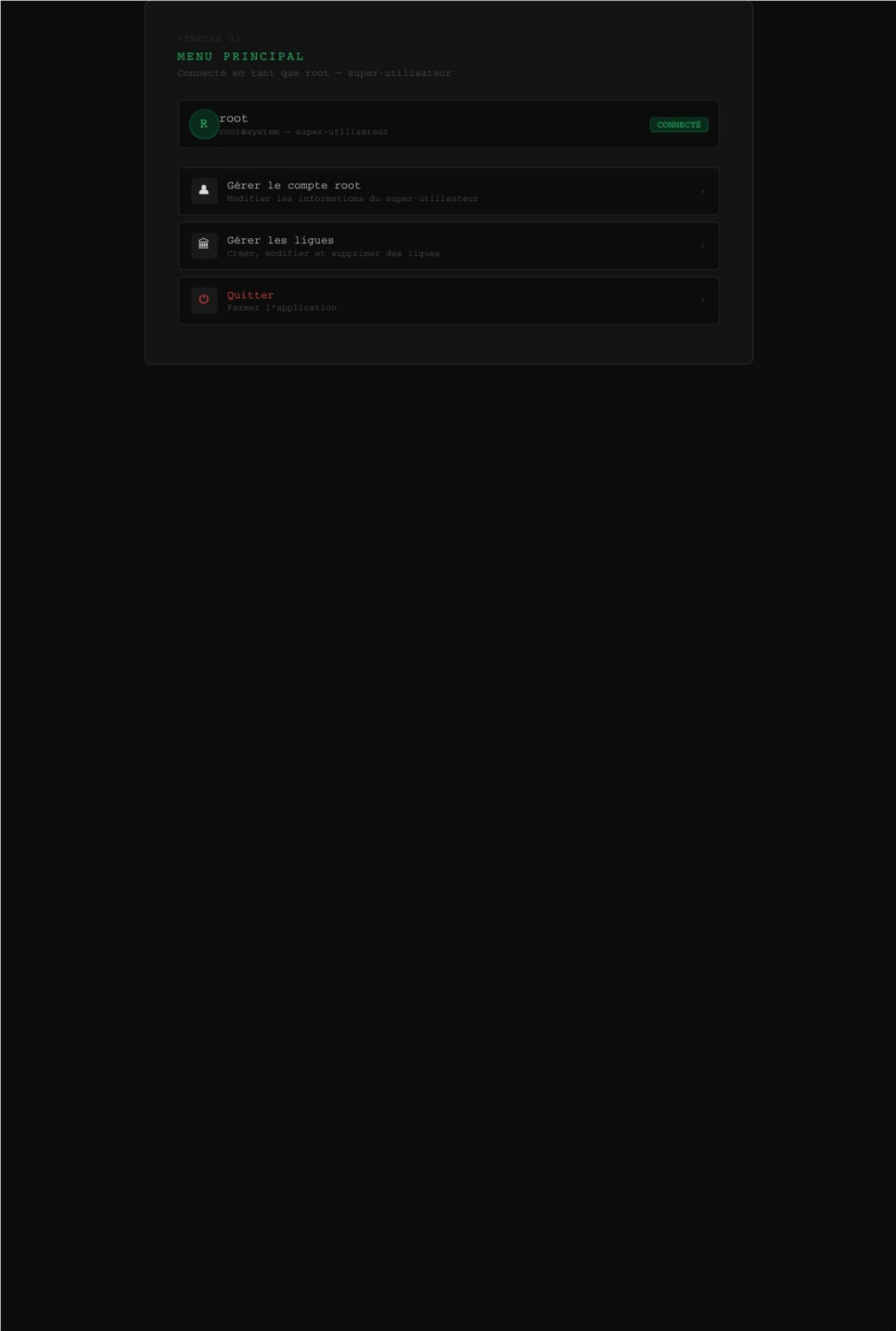
```
mvn test
```

8. Maquettes de l'Interface Graphique

Les maquettes suivantes présentent les 8 fenêtres de l'interface graphique de l'application.



Fenêtre 01 — Connexion



Fenêtre 02 — Menu Principal

FENÊTRE 03

GÉRER LE COMPTE ROOT

Modification des informations du super-utilisateur

Nom

Root

Prénom

Super

Mail

root@system.fr

Password

.....

MODIFIER UN CHAMP

Nouveau nom...

Modifier

Nouveau prénom...

Modifier

Nouveau mail...

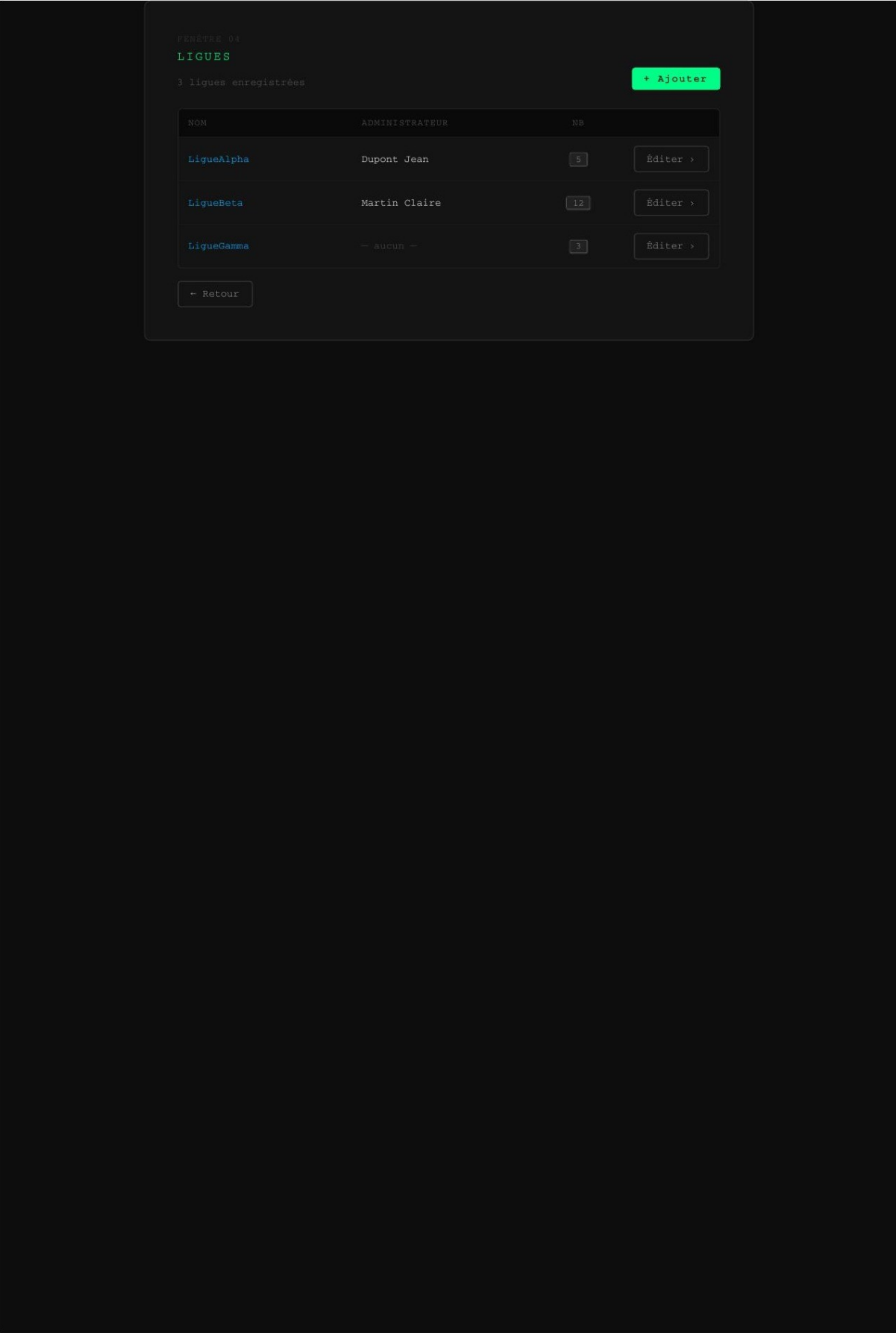
Modifier

Nouveau mot de passe...

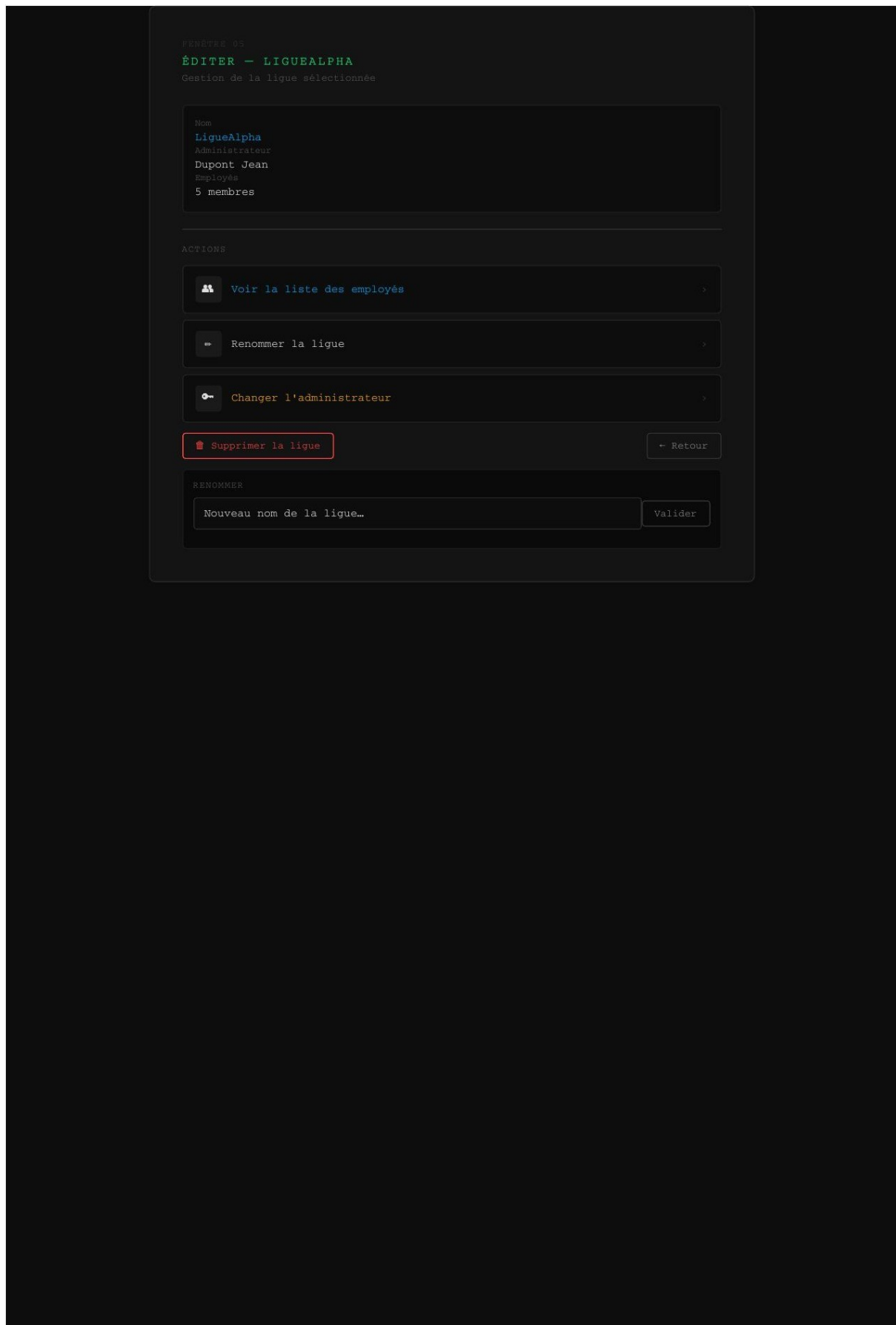
Modifier

← Retour

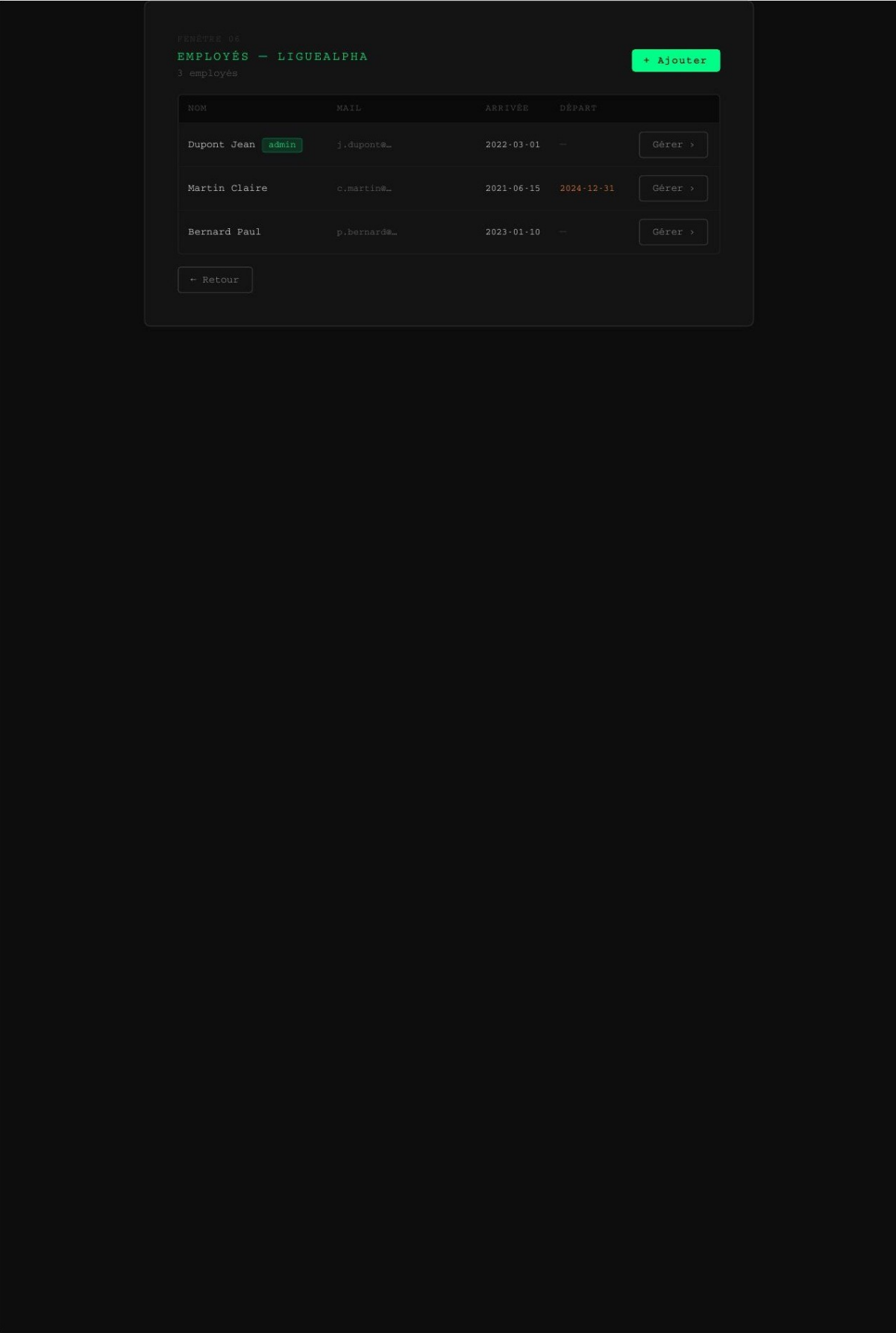
Fenêtre 03 — Gérer le Compte Root



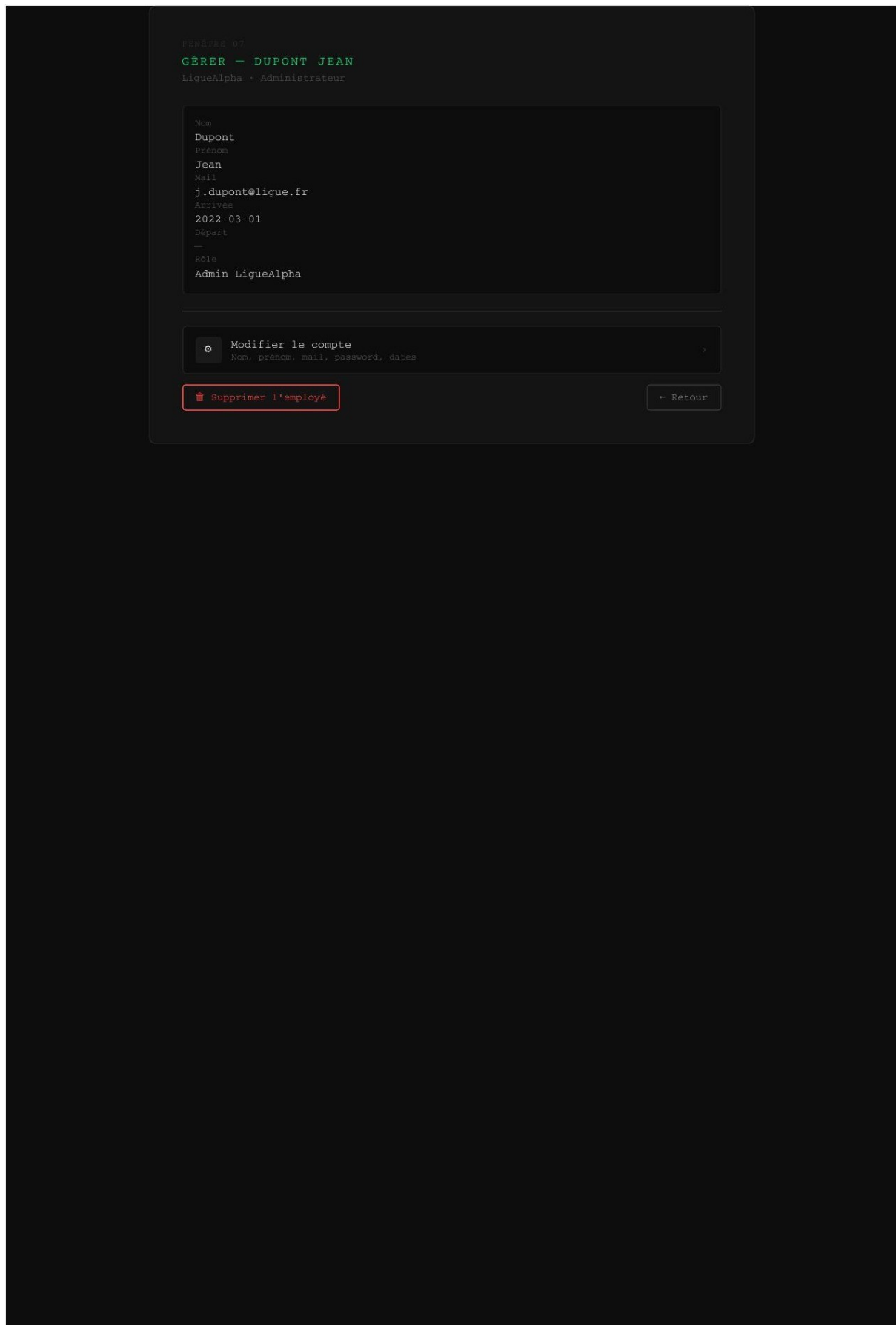
Fenêtre 04 — Liste des Ligues



Fenêtre 05 — Éditer une Ligue



Fenêtre 06 — Liste des Employés



Fenêtre 07 — Gérer un Employé

FENÊTRE 08

MODIFIER LE COMPTE — DUPONT JEAN

Modification des informations de l'employé

NOM

Dupont

✓ Valider

PRÉNOM

Jean

✓ Valider

MAIL

j.dupont@ligue.fr

✓ Valider

MOT DE PASSE

.....

✓ Valider

DATE D'ARRIVÉE

2022-03-01

✓ Valider

DATE DE DÉPART (optionnel)

Laisser vide = en poste

✓ Valider

← Retour

Fenêtre 08 — Modifier le Compte